Instalação e operação (ambiente local)

Documentação técnica - Manual Operacional Digital

Para hospedagem em produção, ver IMPLANTACAO-GOOGLE-CLOUD.md.

Requisitos

- Node.js compatível com o Vite 8 (testado com Node 22 e 24) e npm.
- PostgreSQL (testado com a versão 16) e `psql` para aplicar o schema.
- Opcional: projeto e bucket no Supabase Storage.

1. Banco de dados

Crie o banco e aplique o schema. O script é transacional e pode ser executado de novo sem perder dados.

```
$env:DATABASE_URL='postgres://postgres:senha@localhost:5432/manual_operacional'
psql "$env:DATABASE_URL" -v ON_ERROR_STOP=1 -f backend/sql/schema.sql
```

O próprio `schema.sql` cria a conta inicial (não existe arquivo de seed separado):

- Usuário: `admin`
- Senha: `admin123` (troque no primeiro acesso)

Detalhes das tabelas: Banco de Dados/README.md.

2. Backend

```
cd backend
Copy-Item .env.example .env
npm install
npm run dev
```

Variável	Obrigatória	Descrição
`PORT`	Não	Porta HTTP; padrão `4000`
`DATABASE_URL`	Sim	Conexão PostgreSQL
`JWT_SECRET`	Sim	Segredo de assinatura dos tokens. Sem ele o servidor sobe, mas todo login falha
`FRONTEND_URL`	Sim em produção	Origem permitida pelo CORS; padrão `http://localhost:5173`
`SUPABASE_URL`	Não	URL do projeto Supabase
`SUPABASE_SERVICE_ROLE_KEY`	Não	Chave privada, usada somente no backend
`SUPABASE_BUCKET`	Não	Bucket; padrão `manuals`
`DEV_DEMO_MODE`	Não	`true` usa dados em memória quando o PostgreSQL está indisponível (só desenvolvimento)

Sem Supabase, os arquivos são gravados em `backend/uploads` (pasta ignorada pelo Git). Nunca exponha `SUPABASE_SERVICE_ROLE_KEY` no frontend.

3. Frontend

```
cd frontend
Copy-Item .env.example .env
npm install
npm run dev
```

Variável	Descrição
`VITE_API_URL`	URL do backend vista pelo navegador; padrão `http://localhost:4000`

Acesse `http://localhost:5173`.

Build de produção

```
cd frontend
npm run build        # gera frontend/dist

cd ../backend
npm start
```

O backend **não** serve o `frontend/dist`; ele precisa de um host estático ou proxy reverso.

Verificações

```
cd backend
npm test             # política de senha e nota do questionário
node --check src/app.js

cd ../frontend
npm run build
```

Teste de saúde: `GET http://localhost:4000/health` deve responder `{"status":"ok","service":"Manual Operacional Digital API"}`. Esse teste não verifica o banco.

Modo demonstração

Com `DEV_DEMO_MODE=true`, se o PostgreSQL estiver fora do ar (conexão recusada, senha errada ou banco inexistente), a API usa dados em memória. Os dados somem ao reiniciar, alguns recursos não funcionam (questionário, painel completo) e o modo **nunca** deve ser ligado em produção.

Diagnóstico rápido

Sintoma	Causa provável
401	Token ausente, inválido ou expirado
403	Perfil sem permissão
"Banco de dados indisponível ou credenciais incorretas"	`DATABASE_URL` errada ou PostgreSQL parado
`secretOrPrivateKey must have a value` no login	`JWT_SECRET` não definido
Erro de CORS no navegador	`FRONTEND_URL` diferente da origem real (protocolo e porta incluídos)
Upload local não abre	Permissões da pasta `backend/uploads`
Progresso do PDF não atualiza	Backend não reiniciado ou `schema.sql` não reaplicado